

MIDNIGHT PASSPORT · WITNESS PROTECTION PROGRAM

Portable Passport

Encrypted export and human-recoverable backup for Midnight Passport

DESIGN PROPOSAL · 28 SEPTEMBER 2026

> Design proposal. Not ratified, not implemented, and not security-reviewed. Figures are drawn from the research record in Appendix A; provenance and known gaps are in Appendix B.

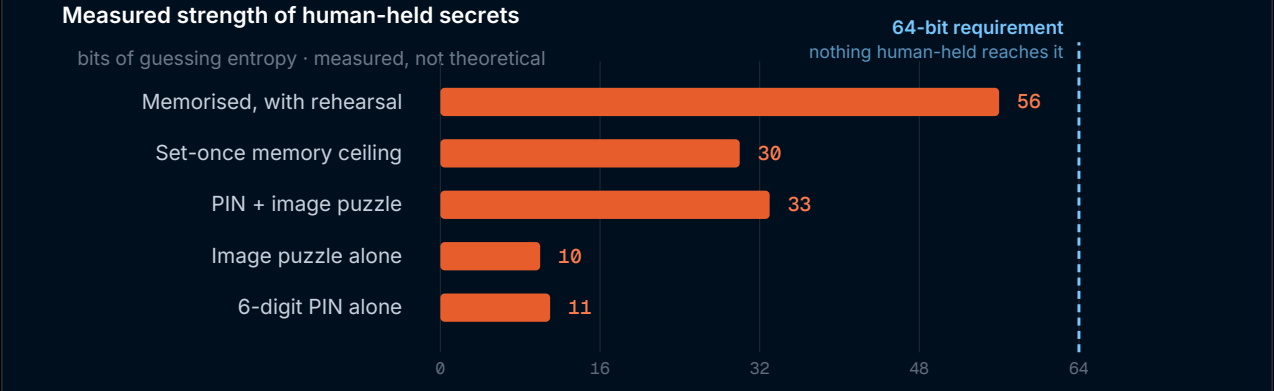
EXECUTIVE SUMMARY

The human cannot hold the key. They can only release it.

Passport links a wallet to a passkey on a phone. We want that identity portable, and the private state the chain cannot rebuild backed up through WPP into the user's own Google Drive, encrypted before it leaves the device. The open question was what an ordinary person can do to decrypt it again.

What the evidence settles

Eight research streams agree on one thing: **no secret a human holds in their head, recognises on a screen, or knows about their own life can carry a 256-bit key.** The measured ceiling is about 56 bits with rehearsal, and below 30 set-once. A key-derivation function buys roughly 25 bits once the ciphertext is downloadable — so a PIN plus an image puzzle is worth about $200 of rented GPU.



256 bits
WebAuthn PRF, biometric-gated
off the scale above · the only factor that clears the bar

What we propose

Three independent release paths, any one of which recovers the backup. The PIN and image puzzle stay — but only inside the device path, where the secure element rate-limits them for free. They never stand alone in front of a downloadable ciphertext.

Path	What the human does	Carries the bits
E1 · Device	Touch the phone	WebAuthn PRF — 256 bits
E2 · Helpers	Ask 2 of 3 trusted people	Threshold shares — no human secret
E3 · Escrow	Scan a printed QR code	128+ bit code — bonus path only

Three things outrank this proposal and do not depend on it: 1. account_custody.compact recover() has no timelock. 2. The contract's own comment says the recovery shares are readable by anyone — “Do not ship.” 3. DeRec is named as the recovery mechanism but sits at v0.0.2 with two archived repos and no audit.

SECTION 1

Summary

Passport links a user's wallet to a passkey on their phone. We want that identity to be portable, and we want the private application state that the chain cannot rebuild to be backed up through the Witness Protection Program (WPP) into the user's own Google Drive, encrypted before it leaves the device. The open question was what set of things an ordinary person can do to decrypt it again.

This proposal answers that question, and the answer inverts the obvious approach. Eight research streams agree that no secret a human holds in their head, recognises on a screen, or knows about their own life can carry a 256-bit key. The measured ceiling for a memorised secret is about 56 bits with permanent rehearsal, and below 30 bits when it is set once and needed later. Personal knowledge questions carry 2.3 to 8.1 bits. Image recognition carries 8 to 12. Meanwhile, once ciphertext sits in a cloud account the attacker can download, a key-derivation function buys roughly 25 bits and nothing else does.

So the human's easy action cannot be **producing** the secret. It has to be **authorising its release**. We propose three independent release paths, any one of which recovers the backup, with the low-entropy human factors placed only where hardware already rate-limits them.

Path	What the human does	Carries the bits
A - Device	Touch the phone (biometric)	WebAuthn PRF, 256 bits
B - Migration	Export the passkey to a new phone	FIDO CXF preserves the PRF seed
C - Helpers	Ask 2 of 3 trusted people	Threshold shares, no human secret at all

A PIN and an image puzzle remain in the design, but only inside Path A, where the secure element enforces the attempt limit for free. They never stand alone in front of a downloadable ciphertext. This is the single change that makes the original idea sound.

> The proposal requires no change to the WPP kernel, the witness package format, the HKDF profile, or the AAD rules. It is additive.

SECTION 2

What was asked for

A phone, with everything linked to a passkey.

Portability, by means of an export.

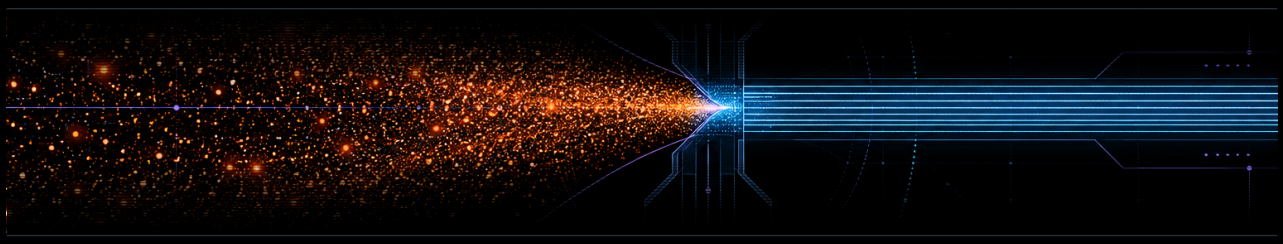
That export backed up via WPP to Google Drive.

Encrypted before it reaches storage.

Decryptable using some set of things a human can easily do.

User experience is the primary constraint, not a secondary one.

Two of these were already satisfied and one was already dangerous. WPP encrypts before upload by construction, and it has completed a live Google Drive round trip. Passkey portability turns out to be solved at the format layer. The danger is that Google is already load-bearing in three separate roles, which Section 3 covers.



SECTION 3

Six findings that constrain any design

F1. No human-held secret reaches 256 bits

Regime	Ceiling	Evidence
With permanent rehearsal	~56 bits	Bonneau & Schechter 2014; 41% still lost it at 16 days
Set once, never rehearsed	below 30 bits	Shay et al. 2012, n=1,476; ~50% failure at 48 hours
24-word BIP39 (256 bits)	never measured	4.5x the largest secret ever studied

Bonneau and Schechter's participants needed a median of 36 logins to learn a 56.4-bit code, and the authors decline to claim it scales past three chunks. In Shay et al., a majority in every condition wrote the secret down, and passphrases were statistically no better than equal-entropy random character strings.

Error tolerance does not rescue this. Only 6% of Bonneau and Schechter's 17-day failures were within edit distance 2 - people lose whole chunks, not characters - so typo tolerance fixes the wrong failure mode.

F2. Recognition and personal knowledge are weaker still

Factor	Theoretical	Measured effective
Biometric	n/a	0 bits - gates an authenticator, produces no stable bits
6-digit PIN	19.9	10.74-11.01
4-of-25 images, ordered	18.2	~8-12
Personal knowledge questions	varies	2.3-8.1 (min-entropy)

Google's analysis of 11 million real recovery claims found "favourite food" broken 19.7% of the time in a single guess, and recall below 80% for every question tested. Their conclusion: it appears next to impossible to find secret questions that are both secure and memorable.

Worse for a wallet, every scheme in this family fails against an attacker who knows the victim. Measured acquaintance success rates run from 10.5% for preference-based questions to 33-39.5% for significant others. None meets the benchmark bar of 1% of accounts per year.

There is a structural reason, and it is not an engineering gap that can be closed. Every one of these schemes is a threshold classifier - "seven of ten, close enough" - not a deterministic function of a secret. The error tolerance that produces the good recall is exactly what makes deterministic key derivation

impossible.

F3. Once the blob is downloadable, the KDF buys about 25 bits

Secret strength	Attacker cost	Verdict
6-digit PIN alone (~11 bits)	cents	no
PIN + image puzzle (~22-33 bits)	~$200, about one day on $1,000 of rented GPU	no
40 bits	~$26,900	marginal
50 bits	~$27.5M	adequate vs most
64 bits	~$451 billion	recommended floor

Required bits, derived: log2(attacker budget in dollars) + 25 + 6 for margin. A $100,000 adversary needs 48 bits. And the mobile constraint means this cannot be tuned away - a one-second Argon2id budget on a phone spans only about 2.1 bits across its entire feasible parameter range.

F4. A progressive web app can reach no hardware entropy source

crypto.getRandomValues is the only randomness primitive available in a browser. TPM 2.0, Apple's Secure Enclave, Android StrongBox, PCIe and USB quantum RNGs, and physically unclonable functions are all unreachable. Cloud QRNG APIs are disqualified because the provider sees the bytes; public randomness beacons because the output is public. WebAuthn's PRF is a deterministic derived secret, not an entropy source.

The Web Cryptography API is explicit that it "provides no lower-bound on the information theoretic entropy present in cryptographically strong random values." A PWA therefore cannot make a standards-backed entropy claim at all. Attestation does not help either: TPM key certification, Android Key Attestation and Apple App Attest all attest key residency, never the entropy consumed.

This is why user-contributed entropy matters here specifically. In a PWA it is the only source available that is independent of the browser's CSPRNG.

F5. Google is already a single point of failure

Role	Principal today
Passkey sync fabric	Google Password Manager / iCloud Keychain
"Add a way back" recovery	Google, Apple, Microsoft, Discord, X, email
Proposed backup storage	Google Drive

WebAuthn associates the PRF with the credential "for the lifetime of the credential", so a synced passkey yields the same wallet seed on every device it reaches. Unit 42's Golden Pass-ta-key attack (August 2026) extracts the Security Domain Secret from Chrome process memory and decrypts all Google-synced passkey private keys. If the backup's key is also Google-reachable, one account compromise takes everything.

WPP already states the invariant that prevents this: provider login never unlocks the WPP root by itself. Enforcing that invariant is the second layer's real job.

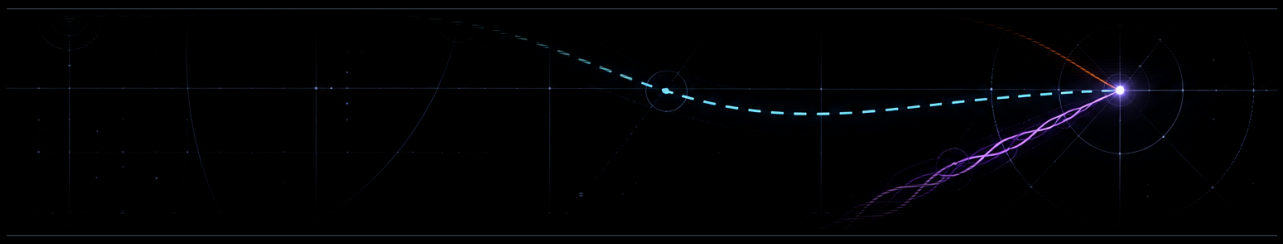
F6. Passkey portability is already solved at the format layer

FIDO's Credential Exchange Format carries PRF state in a Fido2Extensions.hmacCredentials member, and the normative text is emphatic:

> “It is the exporting provider's responsibility to provide the final byte sequence used as the HMAC seed. Regardless of how the credentials enclosed are created, the importing provider MUST store and use these credential as-is during the HMAC operation. There MUST NOT be any additional derivation or domain separators.”

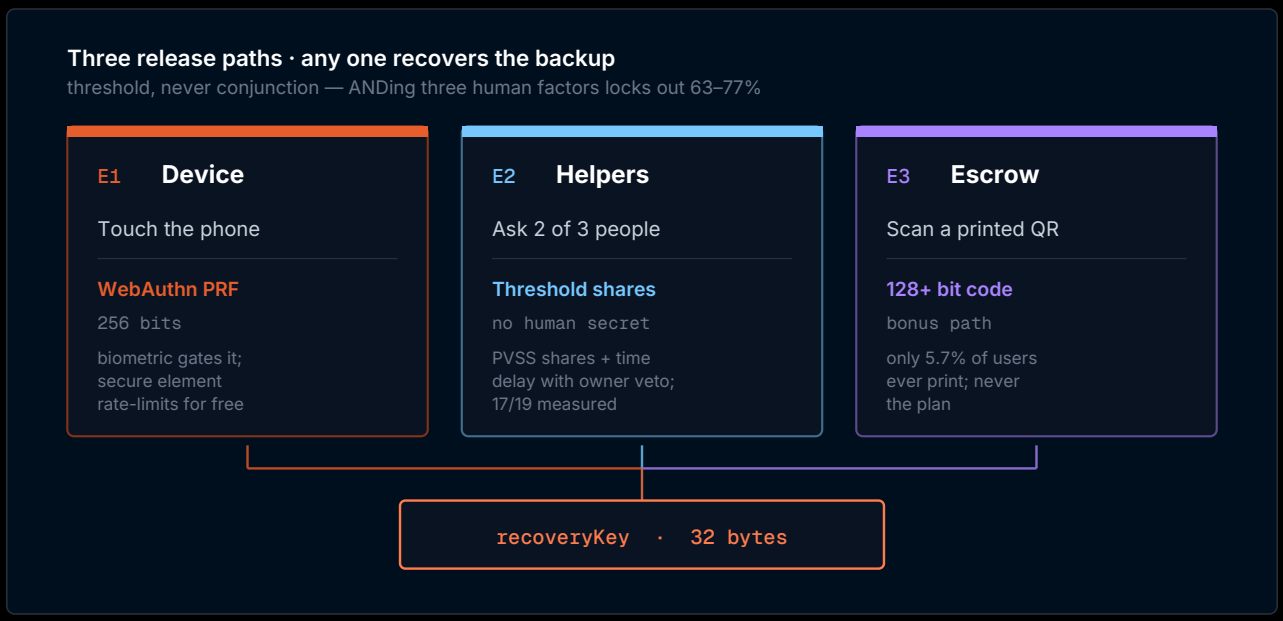
So the passkey-derived wallet seed survives migration. Two caveats: the member is optional in the CDDL, so an exporter may omit it; and while CXF is a Proposed Standard, CXP - the transport that actually moves the bytes - remains a Working Draft. The format is settled; delivery is not.

The consequence for scope is important. WPP does not need to carry the wallet seed. It needs to carry the private application state, which is what it was built for.



SECTION 4

The proposal



4.1 Integration point

WPP's entire public recovery surface is two calls:

```
UnlockedVault.createRecoveryPack(): { wire: Uint8Array, recoveryKey: Uint8Array }
UnlockedVault.fromRecoveryPack(wire, recoveryKey, codecs): UnlockedVault
```

wire is the sealed root record. recoveryKey is 32 random bytes, handed out raw today for a Bitwarden secure note. The second layer is a new artifact that seals those 32 bytes. The kernel need not know it exists.

4.2 Three release paths, not a conjunction

Seal recoveryKey into independent envelopes. Any one opens it. This gives OR at the policy level while each envelope internally requires AND of its own inputs - the same shape as LUKS keyslots or age recipients.

Envelope	Inputs	Rate limiter
E1 Device	WebAuthn PRF (separate credential, UV required) + Argon2id(PIN, puzzle)	Secure element, free
E2 Helpers	Threshold t-of-n shares, encrypted to helper public keys, with proof of correct sharing	Time delay with owner veto

Envelope	Inputs	Rate limiter
E3 Escrow	Printed 256-bit code, QR at error-correction level H	n/a

All three of the originally proposed human factors live in E1, and only there. The PRF carries the security; the PIN and puzzle defend against malware on an unlocked device that can invoke the passkey. They never appear in E2 or E3, so they are never the sole barrier to an offline-attackable blob.

Conjunction is not an option. On Google's measured recovery-channel rates, requiring three human factors together fails 63-77% of the time; even at an optimistic 5% per-factor failure it locks out 14%. The same failure rate on 2-of-3 gives 0.7%. No protocol in the field uses AND, and Vitalik Buterin argues explicitly against 2-of-2.

4.3 The outer wrap must not be verifiable, and must not be co-located

An additional encryption layer over already-encrypted data is sound, but only if the attacker gets no oracle telling them a guess was right. Two things create one: an AEAD tag, and co-location with the payload it protects.

```
wire = aeadEncrypt(recoveryKey, ...) -> Google Drive (unchanged)
wrapped_rk = AES-256-CTR(KEK, recoveryKey) -> NOT Google Drive (no tag)
KEK = HKDF(PRF_biometric || Argon2id(PIN || puzzle))
```

With no tag, every wrong guess yields 32 plausible random bytes and candidates cannot be ranked. Split across trust domains, neither compromise alone yields an oracle. Drive compromise then requires 256 bits; wrapped-key compromise yields indistinguishable bytes; only both together return to the ~22-33 bit problem.

The condition is strict. The wrapped plaintext must be a uniformly random key with zero redundancy - no version byte, no length field, and no downstream AEAD an attacker could use as the check instead. This is RFC 5297's key-wrap rationale. Note that key-committing AEAD does not help here: the tag is a near-perfect verifier either way. Commitment removes a different attack, against a live decryption oracle.

> Consequence for UX: an unauthenticated wrap cannot distinguish "wrong PIN" from "corrupt file". Recovery simply fails. The design must accept that.

4.4 Entropy for the AES key

WPP generates recoveryKey with Node's crypto.randomBytes. Verified from source, that path is randomBytes -> ncrypto::CSPRNG -> RAND_bytes_ex(nullptr, buf, len, 0) -> OpenSSL's public per-thread CTR-DRBG. Two observations: it requests strength 0, and it uses the public DRBG rather than RAND_priv_bytes, which is the one OpenSSL intends for long-term private key material. SP 800-90C also warns that reseeding a DRBG from another DRBG does not guarantee prediction resistance, which is exactly this chain.

The threat is not the OS CSPRNG. No documented case exists of getrandom, BCryptGenRandom or SecRandomCopyBytes producing weak output on a properly-booted machine. Every wallet loss on record happened because software never reached the CSPRNG or discarded its output - Milk Sad used Mersenne Twister seeded with 32 bits of clock; Randstorm fell back to Math.random; Coldcard's 2026 build-config error silently substituted a software PRNG, draining about 1,816 BTC (~$116M) from over 5,200 addresses.

That incident is also the best argument for user entropy, because Coinkite's own advisory exempts it:

> “If you entered at least 50 fair and independent rolls, and the rolls were not recorded or exposed, we do not consider the resulting seed at risk from this RNG issue alone.”

We therefore propose: **concatenate all available sources and run them through a vetted conditioner**, always, with no branch that can select fewer. This makes user entropy a floor rather than an average - the topology that rescued Coldcard's dice users. Coldcard hashes device and user values together; Trezor mixes host and internal randomness via SHA-256. Replace-mode, where dice are the sole source, is rejected: no measured bias data exists for consumer d6 dice.

> Concatenate, never XOR. Dodis et al. show that refresh(S, I) = S XOR Extract(I), given independent and unbiased inputs, still pins the state to two possible values forever - "the refresh procedure failed to accumulate more than 1 bit of entropy". Concatenate-then-hash is a split-key PRF in the random-oracle model; XOR-then-PRF is not. NIST SP 800-90B section 3.1.6 standardises exactly this: concatenate extra noise sources into the conditioner, require a vetted conditioning component, and credit zero entropy to the extras. And per RFC 5869 section 3.4, untrusted sources belong in the IKM slot, never in the salt - salt "must not be chosen or manipulated by an attacker".

Source	Contribution	Availability
Browser CSPRNG	Always mixed	Everywhere
Optional user dice	100 casino d6 = 258.5 bits	Opt-in, any tier
Native TRNG / TPM	Mixed where present	Native companion only

Note that 99 rolls yields 255.91 bits - just under the target - and both Coldcard and SeedSigner use 99. Specify 100. Reject single card decks: a shuffled 52-card deck carries only 225.6 bits, and some implementations silently ignore the last dozen cards.

Two sources to rule out explicitly. Public randomness beacons contribute exactly zero bits of secrecy - NIST's own page warns "DO NOT USE BEACON GENERATED VALUES AS SECRET CRYPTOGRAPHIC KEYS" - though they remain useful for proving a key was not fixed before a given time. And a verifiable random function cannot help either: RFC 9381 section 3.4 states its output is "always distinguishable from random by any party that knows a valid VRF proof", so verifiability and key-secrecy are mutually exclusive by construction.

Note also that dice are the only candidate source whose entropy is provable rather than estimated. Measured human-derived alternatives are worth single digits: Linux keyboard and mouse timing yields 0.68 bits of min-entropy per event, and 69 million Yahoo passwords measured 6.5 bits.

> A normative hook exists, and it is German, not American. BSI AIS 31 NTG.1.4 requires "at least 240 bits of min-entropy from at least two different noise sources each", and that those sources "shall employ different principles". NIST is permissive throughout.

4.5 Nonces and hedging

freshRandom also produces every 12-byte GCM nonce and every generationId. An RNG weakness there is worse than a key-guessing problem, because a single nonce reuse under GCM recovers the authentication key outright. Widening the nonce does not fix this: a 192-bit nonce buys about 48 bits of message count, but a stuck RNG collides with probability 1 at any width.

We propose hedged derivation instead, deriving key and nonce from a long-term secret, the message, and whatever randomness is available, so that total RNG failure degrades to deterministic-but-secure rather than catastrophic. FIPS 204's rationale is directly citable: for this purpose, even a weak random bit generator may be preferable to the fully deterministic variant.

Do not rely on statistical self-tests. SP 800-90B's two continuous tests run on the raw noise source, and validated modules use 1,024 samples; 32 bytes is far below that. Debian's 15-bit-entropy OpenSSL passed every statistical test ever applied to it. The checks that actually pay are duplicate-output detection across calls and fork or boot-id detection, since process-state duplication is likelier in practice than entropy starvation.

SECTION 5

The human experience

UX is the primary constraint, so this section is the specification and Section 4 is its implementation.

Moment	What the user does	What happens underneath
Enrol	Touch sensor. Name two helpers. Print one sheet.	E1, E2, E3 all created. No skip affordance.
Normal backup	Nothing.	State encrypted locally, uploaded to their Drive.
New phone	Export passkey to the new device.	CXF carries the PRF seed; E1 still opens.
Phone lost, sheet held	Scan the QR.	E3 opens recoveryKey directly.
Phone and sheet lost	Ask two helpers. Wait out the delay.	E2 reconstructs; owner devices can veto.
Yearly	Confirm a prompt.	Heartbeat verifies helpers still hold valid shares.

Four design rules the evidence supports

No skip affordance on recovery setup. A participant offered "Remind me later" concluded that skippable things are not important. This is the one evidenced anti-pattern in the wallet-UX literature.

Replace the warning with a mechanism. Comprehension survives a full guided session, risk-aware users still lose the paper, and stake size does not change behaviour. Warnings do not work; heartbeats and forced verification might.

Show the user who their helpers are. In the only measured guardian-recovery study, 17 of 19 participants succeeded, but they had to be reminded who their trustees were. Helper-side verification must also not rely on recognising a voice - phone impersonation by socially-close attackers succeeded in that study.

Do not rely on the user printing anything. Of 32,784 users who displayed a recovery code, only 1,873 (5.7%) printed it; the largest group copied it to the clipboard. Only 14.8% stored it in two or more places. E3 is a bonus path, never the plan.

One asset worth designing toward

The dominant mental model users bring to a wallet is the password, which is why they expect a reset. For a seed-phrase wallet that expectation is wrong and dangerous. For a passkey wallet with synced credentials

it is correct. That is a rare alignment between naive expectation and system reality, and it should be designed toward rather than educated away.

The counterweight, from 28 expert interviews: the single biggest obstacle to FIDO2 adoption is fallback and recovery, with unclear handling of lost or stolen devices. Experts are split on whether synced passkeys solve recovery at all. That is precisely the gap this proposal fills.

> The cleanest framing for the recovery copy: Google can reissue backup codes because they only gate authentication. Apple cannot reissue the recovery key because it gates encrypted data. A wallet backup is in Apple's category - regeneration is not available as a safety net.

SECTION 6

Work items

Independent of this proposal, and higher priority than it

#	Item	Why
1	Add a timelock and owner veto to account_custody.compact recover()	It currently bumps device_epoch and registers a new device immediately. Argent's precedent is 1-3 days, cancellable.
2	Replace the plaintext ledger shares with a PVSS scheme	The contract's own comment: anyone can reconstruct the recovery secret. Do not ship.
3	Treat DeRec as unavailable and assign an owner	lib-derec is 0.0.2 and self-describes as pre-release; the spec repo has not moved since June 2025; two repos are archived; no audit found. The published architecture names it as the recovery mechanism.

This proposal

#	Item
4	Enrol a second, separate credential for WPP recovery rather than reusing the wallet PRF credential.
5	Implement the three envelopes with an unauthenticated outer wrap and enforced storage separation.
6	Implement hash-mix entropy with no selectable branch, plus optional 100-roll dice input.
7	Add commit-then-reveal verification of the user's entropy contribution - two-party joint random secret generation, which is precisely what Trezor's shipping entropy-check protocol implements. Distributed key generation is the wrong tool: there is no g^x for a symmetric key, so its commitment machinery has nothing to verify.
8	Add duplicate-output and fork/boot-id detection around the key path.
9	Move to hedged key and nonce derivation.
10	Enforce the correlation rule at enrolment and whenever a way back is added.

#	Item
11	Measure Argon2id on the real target device. A recovery flow can justify 10-60 seconds, worth about 3.5 extra bits.

Already correct, and worth not regressing

Passport does not call PublicKeyCredential.toJSON(), which would ship the PRF-derived key to its own server. PRF output is read directly, consumed locally, with 16 zeroization sites.

The two derivations are domain-separated with a documented rationale.

largeBlob is requested as "preferred" with absence handled - correct, since CTAP 2.1 mandates only 1,024 bytes for the entire array, shared across all credentials.

WPP's threat model already refuses forward-secrecy and cryptographic-erasure claims, and already requires that an authenticated backup may be stale.

SECTION 7

What this proposal does not claim

Not AAL3. NIST SP 800-63B-4 section 2.3.2: syncable authenticators SHALL NOT be used at AAL3. A PRF-derived seed from a syncable passkey cannot reach it.

Not hardware non-exportability. The wallet seed is derived, not hardware-held; it materialises in memory on every unlock.

Not a standards-backed entropy claim in the PWA tier. WebCrypto gives no lower bound, and SP 800-90C's full-entropy classes require a physical entropy source. A software-only wallet tops out at RBG2(NP).

Not an HSM escrow service. Apple's own top tier abandons that pattern; 2025-26 DRAM-interposer attacks (Battering RAM, WireTap, TEE.fail) break attestation on SGX, TDX and SEV-SNP; and the state of the art, Signal's SVR3, is a three-trust-domain distributed build.

Not forward secrecy or cryptographic erasure. Deterministic recovery deliberately retains access under retained roots.

SECTION 8

Open decisions

Does E1 require a credential with backupEligible = 0, or is a synced fallback allowed with a downgraded claim? Note there is no authenticatorSelection knob to require device-bound; the relying party can only inspect the flag after create() and refuse.

Helper set for E2: human helpers, user-owned devices, or an institutional option?

Is the image puzzle mandatory in E1, or an opt-in alternative to a longer PIN?

Is codex32 worth the delivery cost? It error-corrects by hand and has Shamir built in, but the only JavaScript implementation has a single GitHub star, and an unaudited dependency in the key path is the exact pattern that killed Milk Sad and Randstorm.

Does a duress mode get built? Listed, not recommended - completeness is unfalsifiable to a coercer.

APPENDIX A

Evidence

Every load-bearing figure in this proposal, with its source. Cost figures in section 3 are derived arithmetic over cited anchor benchmarks and are marked as such in Appendix B.

Claim	Figure	Source
Memorised secret ceiling	56.4 bits; 94% recall in use, 59% at median 16 days	Bonneau & Schechter, USENIX Security 2014
Set-once recall	~30 bits; 47-58% recall at 48 hours, n=1,476	Shay et al., SOUPS 2012
Personal knowledge min-entropy	2.3-8.1 bits; favourite food 19.7% in one guess	Bonneau et al., WWW 2015 (11M claims)
Graphical secret effective entropy	G-tilde(0.5) = 8.06 bits vs 18.57 theoretical; 21% in 20 guesses	Aviv et al., arXiv 1811.10548, Table 9
Passfaces bias	10% of male users cracked in 2 guesses	Davis, Monrose & Reiter, USENIX Security 2004
Argon2id attacker throughput	1,703 H/s at m=64MiB, t=3 on RTX 4090 (measured anchor)	hashcat v7.0.0 release notes
Guardian recovery success	17 of 19 participants authenticated successfully	Schechter, Egelman & Reeder, CHI 2009
Recovery-code print rate	1,873 of 32,784 (5.7%); 24% of recovery attempts lost the account	Holtervennhoff et al., USENIX Security 2024
Recovery method comparison	Printed codes fastest setup (1.0s) slowest use (28.0s); U2F reverse, lowest SUS	Reese et al., SOUPS 2019, n=72
Seed-phrase recognition	43% could correctly identify an image of a seed phrase, n=643	CHI 2025
Prior wallet losses	22.5% had already lost bitcoins; majority could not recover	Krombholz et al., Financial Cryptography 2016, n=990
Coldcard RNG failure	~1,816 BTC (~$116M) from 5,200+ addresses; dice users exempted	TRM Labs; Coinkite advisory, 2026
Synced passkey compromise	Security Domain Secret extracted from Chrome memory	Palo Alto Unit 42, Golden Pass-ta-key, 3 Aug 2026
AAL3 exclusion	Syncable authenticators SHALL NOT be used at AAL3	NIST SP 800-63B-4 section 2.3.2, final 26 Aug 2025
PRF continuity on export	Importing provider MUST use credentials as-is; no additional derivation	FIDO CXF v1.0 Proposed Standard
Multi-source requirement	240 bits min-entropy from two sources employing different principles	BSI AIS 31 NTG.1.4, v3.0, 10 Sep 2024
Argon2id parameters	2 GiB/t=1/p=4 or 64 MiB/t=3/p=4, 128-bit salt, 256-bit tag	RFC 9106
Key commitment	Partitioning-oracle attacks; plain AES-GCM-SIV not key-committing	Len, Grubbs & Ristenpart, USENIX Security 2021
Brain wallets	884 wallets found, all but 21 drained, usually within 24 hours	published brain-wallet research
XOR combiner refuted	Independent unbiased inputs still accumulate under 1 bit of entropy	Dodis, Pointcheval, Ruhault, Vergniaud & Wichs, CCS 2013
Standardised combiner	Concatenate extra noise sources into a vetted conditioner; credit them zero entropy	NIST SP 800-90B section 3.1.6
Untrusted input placement	Salt "must not be chosen or manipulated by an attacker"	RFC 5869 section 3.4
Public beacons	"DO NOT USE BEACON GENERATED VALUES AS SECRET CRYPTOGRAPHIC KEYS"	NIST Randomness Beacon

Claim	Figure	Source
Human input entropy	0.68 bits min-entropy per keyboard/mouse event	Lacharme, Linux RNG analysis
QRNG necessity	"Quantum random number generators solve no real issue with our current hardware RNGs"	Ericsson Security Research, arXiv 2112.00399

APPENDIX B

Provenance and caveats

Scope of this review

This note reads source code, documentation, published specifications and a minified production bundle. It is not a live run, not a cryptographic review, and nothing in it has been tested.

The custody contract

The account_custody.compact findings come from contracts-stagenet in the demo repository. The shipped client references ACCOUNT_CUSTODY and device_epoch but not recovery_shares or recover(), so it does not appear to exercise the share path. Treat item 2 as a design-review finding, not a live vulnerability.

Figures withdrawn during review

Two attributions were checked and dropped. A "5.19 bits of effective entropy" figure commonly attributed to Bonneau, Preibusch & Anderson does not appear in that paper (the actual figures are 12.90/12.83/12.56 bits average-case). And a "1 to 1.2 bits per keystroke" figure measures information leaked to an attacker, not entropy content. Neither should be published.

No single theorem covers the combiner

There is no published theorem in the exact form "combiner C is secure if at least one of N sources has conditional min-entropy k". Section 4.4's claim is assembled from split-key PRF results and Dodis et al.'s legitimacy condition, and should be stated that way rather than cited to a single source. Note also that Coretti et al. (ITC 2025) require all sources to be independently unpredictable and do not establish the weaker claim.

Figures that must not be asserted

Signal's production PIN attempt limit, WhatsApp's permanent attempt threshold, Apple's recovery-key length, and a "39 methods" passkey-attack figure could not be sourced and are excluded. A "1.4M BTC / $1B" Randstorm figure is secondary press, not in the original report. An "81% SMS / 75% email" figure circulating without attribution was also excluded.

A caught fabrication

An automated PDF extraction produced alpha-guesswork values of 11.7 / 12.5 / 13.6 bits where direct text extraction of the same table showed 5.54 / 6.85 / 8.95. Only verified figures appear here. Any table sourced by automated extraction should be re-checked against the PDF text.

Search coverage

The research exhausted its web-search budget partway through; later work used direct fetches. SOUPS 2023-2026 and Financial Cryptography 2022-2026 were not swept systematically, and no controlled study of wallet-recovery success rates exists in the literature at all - that is a genuine hole in the field, not a search failure.

Derived versus measured

Cost figures in F3 are derived arithmetic over cited anchor benchmarks, not direct measurements. The mobile Argon2id figure rests on a single unverified data point and should be replaced with a first-party measurement on the real target device.